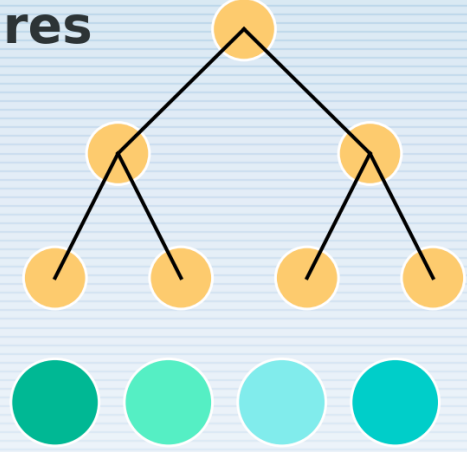
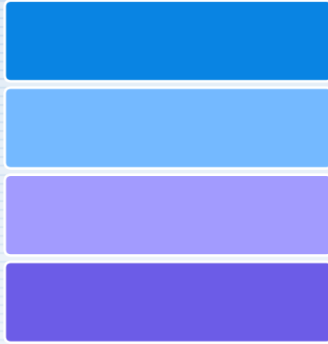




Data Structures



ملخص هياكل البيانات

Data Structures - Simplified Summary

ملخص مبسط وشامل لمادة هياكل البيانات

من المحاضرات 1، 2، 4، 5 و 6

مع رسوم توضيحية إبداعية وجداول تعقيد شاملة

الفصل الأول: المقدمة ولغة الكفاءة

Introduction & Efficiency

ما هي هياكل البيانات؟

بكفاءة. ثلاث كلمات تلخصها بالكامل: نظم (ترتيب منطقي للبيانات)، خزن (في الذاكرة)، وكفاءة (سرعة أعلى وذاكرة أقل). هياكل البيانات هي طريقة متخصصة لتنظيم البيانات ومعالجتها واسترجاعها وتخزينها

لماذا نحتاج هياكل البيانات؟

بدون هياكل بيانات مناسب:

- بحث بطيء: قد تفحص كل عنصر واحدا تلو الآخر
- تحديث صعب: الإدراج والحذف يصححان فوضى
- هدر في الذاكرة: تخصيص غير ذكي للمساحة

ثلاثة هياكل أساسية بتشبيهات يومية

- المكس Stack: كومة أطباق - آخر داخل أول خارج (LIFO)
- الطابور Queue: طابور البنك - أول داخل أول خروج (FIFO)
- المصفوفة Array: صف خزائن مرقمة - لكل عنصر مكان ثابت

النوع المجرد للبيانات ADT

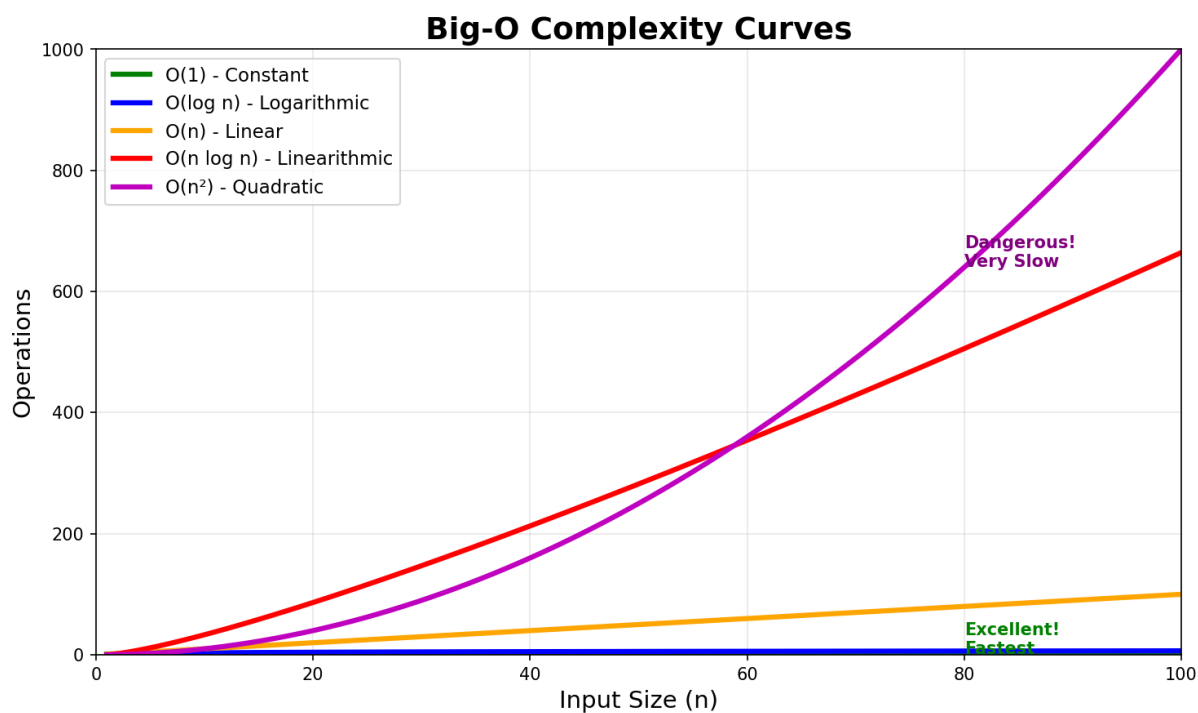
ال ADT هو الفكرة: ماذا يفعل هذا النوع؟ أما هيكل البيانات فهو التنفيذ: كيف نبني تلك الفكرة في الذاكرة؟

التشبيه: أضرار الريموت كنترول - أنت تعرف وظيفة كل زر دون أن تحتاج لمعرفة الدائرة الكهربائية داخل الجهاز.

ترميز Big-O

رياضي يصف أداء الخوارزمية في أسوأ حالة. لماذا الأسوأ؟ لأن معرفة الحد الأسوأ تضمن موثوقية البرنامج مهما كان شكل المدخلات. ترميز

الترميز	المعنى	مثال
$O(1)$	ثابت	الوصول لفهرس مصفوفة
$O(\log n)$	لوغاريتمي	البحث الثنائي
$O(n)$	خطي	بحث عنصرا عنصرا
$O(n \log n)$	خطي لوغاريتمي	Merge Sort
$O(n^2)$	تربيعي	فرز الفقاعات



ملاحظة مهمة:

عند مليون عنصر، الخوارزمية الخطية تنفذ مليون عملية بينما اللوغاريتمية نحو 20 عملية فقط!

الفصل الثاني: الهياكل الخطية

Linear Data Structures

المصفوفة Array

عناصر مخزنة بترتيب ومتاحة عبر الفهرس. في المصفوفة [10, 20, 30, 40] القيمة 10 تقع في الفهرس 0 والقيمة 30 في الفهرس 2. مجموعة

التعقيد	كيف تتم؟	العملية
$O(1)$	مباشرة عبر الفهرس	الوصول
$O(n)$	فحص عنصر عنصر	البحث
$O(n)$	إزاحة العناصر لفتح مكان	الإدراج
$O(n)$	إزاحة العناصر لسد الفراغ	الحذف

ملاحظة مهمة:

لماذا الوصول $O(1)$ ؟ لأن عناصر المصفوفة كتلة ذاكرية متصلة كعربات قطار متلاصقة، فيحسب الحاسوب موقع أي فهرس بمعادلة مباشرة.

المكدس Stack - LIFO

هيكل خطي تتبع عملياته ترتيب LIFO (آخر داخل أول خارج):

- إضافة عنصر للقمة: Push
- إزالة عنصر القمة: Pop
- مشاهدة القمة دون إزالتها: Peek

مثال بلغة C#:

```
Stack<int> stack = new Stack<int>();
stack.Push(10);
stack.Push(20);
Console.WriteLine(stack.Peek()); // 20
stack.Pop();
```

الطابور Queue - FIFO

كطابور انتظار حقيقي (أول داخل أول خروج):

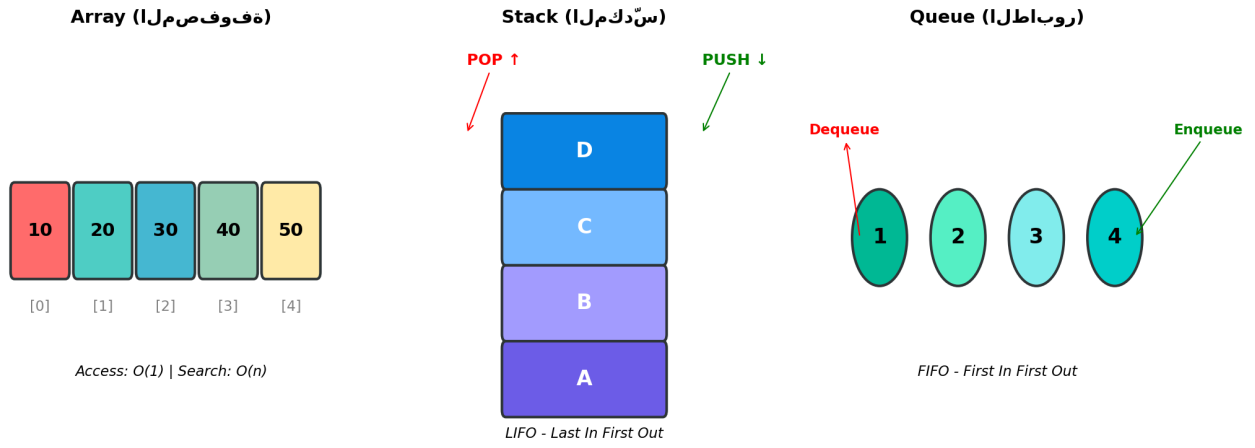
• Enqueue: إضافة للعنصر في مؤخرة الطابور

• Dequeue: إزالة العنصر من المقدمة

• Front/Ppeek: عرض أول عنصر

أداؤها ممتاز: Enqueue و Dequeue كلاهما $O(1)$

التطبيقات: أنظمة الجدولة، أنظمة التشغيل، وخوارزمية BFS.



الفصل الثالث: العودة Recursion

Recursion

ما هي العودة؟

تقنية برمجية تستدعي فيها الدالة نفسها، وتقسم المشكلة إلى نسخ أصغر حتى تصبح بسيطة الحل مباشرة.

تشبيهات من الواقع

- مرآة تقابل مرآة - انعكاس لا نهائي
- الدمى الروسية المتداخلة - دمية داخل دمية
- مجلد داخل مجلد - بحث متداخل
- أغصان شجرة تتفرع لأغصان أصغر

ركنا العودة

- حالة الأساس Base Case: شرط التوقف - اللحظة التي تصبح المشكلة بسيطة
- الحالة العودية Recursive Case: الدالة تعمل على نسخة أصغر وتقترب من حالة الأساس

مثال: حساب المضروب Factorial

$$\text{factorial}(5) = 5 \times \text{factorial}(4) = 5 \times 4 \times \text{factorial}(3) = \dots = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

عند 1 (factorial(1) تتحقق حالة الأساس فيبدأ الإرجاع بالتصاعد.

ملاحظة مهمة:

الخطر: بلا حالة أساس تستمر الدالة في استدعاء نفسها حتى ينهار البرنامج بانفجار المكس Stack Overflow!

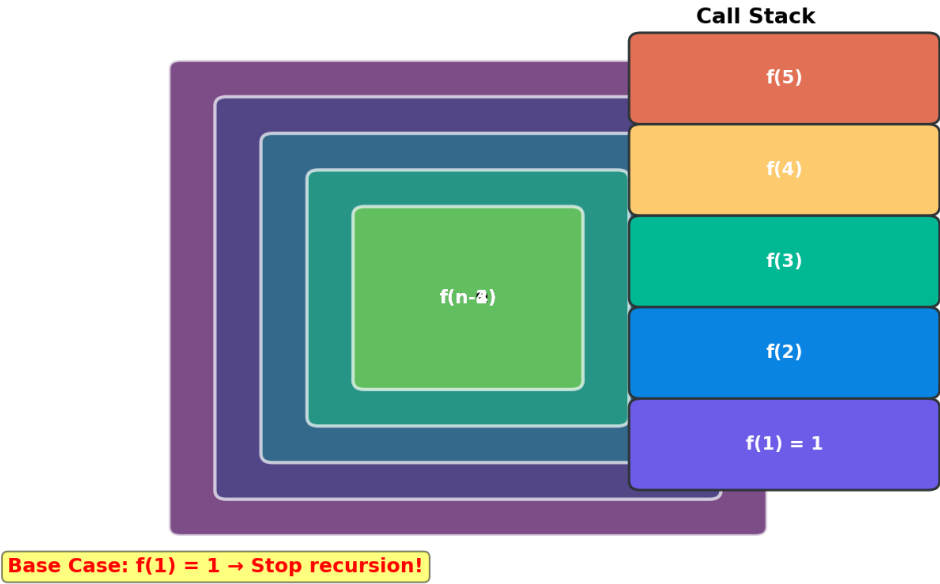
القواعد الذهبية الثلاث

- يجب أن توجد حالة أساس دائما
- يجب تقليص حجم المشكلة في كل استدعاء
- يجب أن يتحرك كل استدعاء نحو شرط التوقف

العودة مقابل التكرار

المعيار	العودة	التكرار
الأنسب لـ	الشجرات والهرميات	المهام الخطية
طول الكود	أقصر وأوضح	أطول غالبا
الذاكرة	تستهلك أكثر	أوفر

Recursion (دَوْعِلَا)



الفصل الرابع: الفرز والبحث

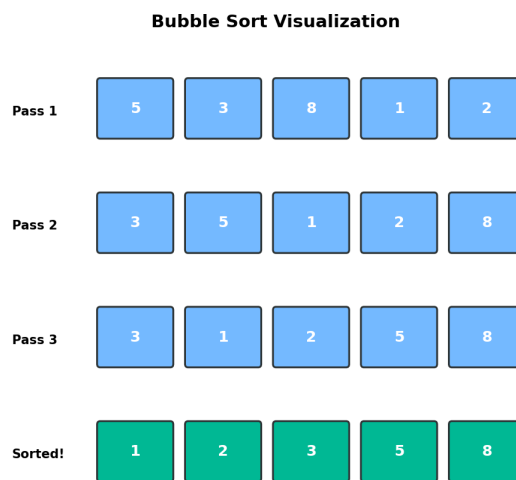
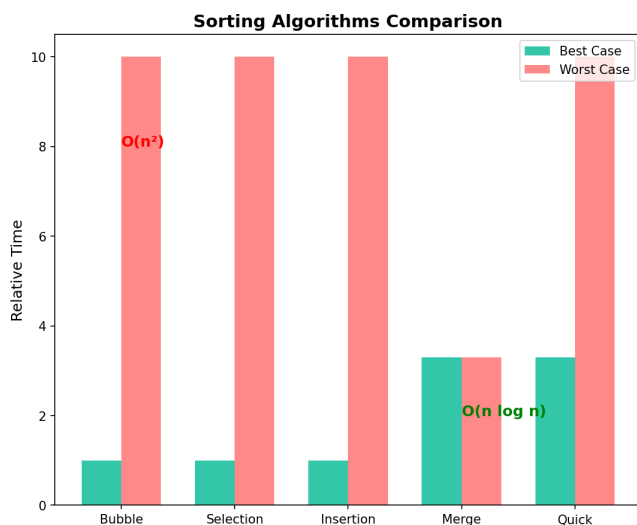
Sorting & Searching

خوارزميات الفرز الخمس

الخوارزمية	الفكرة	التعقيد	الذاكرة
Bubble	قارن الجارين وابدل	$O(n^2)$	$O(1)$
Selection	الأصغر للبداية	$O(n^2)$	$O(1)$
Insertion	أدخل بموضعه الصحيح	$O(n^2)$	$O(1)$
Merge	قسم، رتب، ادمج	$O(n \log n)$	$O(n)$
Quick	محور وقسم	$O(n \log n)^*$	$O(\log n)$

ملاحظة مهمة:

Quick Sort دائما مقابل ذاكرة $O(n \log n)$ تضمن Merge Sort. $O(n^2)$ الأسرع عمليا، لكن أسوأ حالاته Quick Sort.



البحث الخطي Linear Search

- افحص العناصر واحدا واحدا
- التعقيد: $O(n)$
- لا يتطلب ترتيبا مسبقا

البحث الثنائي Binary Search

- يتطلب مصفوفة مرتبة
- قارن المنتصف ثم تخلص عن نصف العناصر
- التعقيد: $O(\log n)$
- في مليون عنصر يحتاج نحو 20 مقارنة فقط!

Binary Search (البحث الثنائي)



Each step eliminates HALF the remaining elements → $O(\log n)$
 1,000,000 elements → only ~20 comparisons needed!

البحث في الرسوم البيانية

- $O(V+E)$ - العرض): يستكشف مستوى مستوى - يستخدم الطابور) BFS
- $O(V+E)$ - العمق): يغوص في فرع حتى نهايته - يستخدم المكس) DFS

الفصل الخامس: ورقة الغش

Exam Cheat Sheet

أهم النقاط

- الوصول لعنصر المصفوفة $O(1)$ لأن الذاكرة متصلة
- المكدس LIFO والطاير FIFO - لا تخلطهما أبدا
- العودة بلا حالة أساس = Stack Overflow
- البحث الثنائي لا يعمل إلا على بيانات مرتبة
- Merge Sort: $O(n \log n)$ مقابل ذاكرة $O(n)$ ضمان
- Quick Sort: $O(n^2)$ الأسرع عمليا وأسوأ حالاته
- مع المكدس DFS مع الطاير و BFS

جدول التعقيد الشامل

الهيكـل	الوصول	البحث	الإدراج	الحذف
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Stack	-	$O(n)$	$O(1)$	$O(1)$
Queue	-	$O(n)$	$O(1)$	$O(1)$

الخوارزمية	أفضل	أسوأ	الذاكرة
Bubble	$O(n)$	$O(n^2)$	$O(1)$
Selection	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion	$O(n)$	$O(n^2)$	$O(1)$
Merge	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Binary Search	$O(1)$	$O(\log n)$	$O(1)$

كيف تختار في الامتحان؟

إذا سئلت عن أقل ذاكرة: Bubble أو Selection أو Insertion لأنها $O(1)$

إذا سئلت عن ضمان السرعة: Merge Sort هي الجواب الآمن

إذا سئلت عن الأسرع عمليا: Quick Sort في أغلب الحالات